

CKMRsim: A Population level analysis

Paige Gardner, UC Santa Cruz pogardne@ucsc.edu

CKMRsim: A Population level analysis

The last ckmr script analysed the whole data set at once. While this didn't initially appear to be a problem, we want to assume that each population is isolated from one another. As a result, we need to run ckmr again on each population.

1. Load in libraries and data
2. Create new data frames for each watershed/population
3. Run CKMR code on each population.

1. Prep library and data

```
library(tidyverse)
library(MCMCpack) # for animal model
library(ggplot2) # plots
library(here) # easy data file access for Git
library(CKMRsim)
```

```
# Phenotype and metadata
ctmax <- read.csv(here("Genomics/Data/Omy5_inversion_metadata_merge.csv"))

# SNP data -----
# Data received 7/2/26
snp <- read.csv(here("Genomics/Data/PaigeAdapCapFishSNPGenotypes.raw.csv"))
# extract first 7 characters into column with the same name as the ctmax dataframe
str(snp$Sample_ID)
snp$indiv.ID <- substr(snp$Sample_ID,1,7)
# remove original sample_ID column
snp <- snp %>%
  dplyr::select(-Sample_ID)
```

```

snp <- snp %>% relocate(last_col())

### rename the columns -----
# make the periods be _ for the next steps.
snp <- snp %>%
  rename_with(~ gsub(".", "_", .x, fixed = TRUE))
# make note of the current order of loci in the data set
nc <- ncol(snp)
loci <- str_replace( # expects string, pattern, replacement
  names(snp)# returns the names of the columns
  [seq(2, nc, by = 2)], # creates a sequence of even numbers starting at 2 and ending at nc,
  "\\.\.\.\.[0-9]+$", "") # this cleans up column names. So I don't think this will impact my

# reset the locus names
names(snp)[seq(2, nc, by = 2)] <- str_c(loci, "1", sep = ".") #put a .1 at the end of the first
names(snp)[seq(3, nc, by = 2)] <- str_c(loci, "2", sep = ".") # put a .2 at the end of the second
snp <- snp %>%
  rename(indiv.ID = indiv_ID)

# MHap data-----
## Neutral
neutral_mhap <- read.csv(here("Genomics/Data/Neutral_genotypes_mhapcode_wide.csv"))
nc <- ncol(neutral_mhap)
loci <- str_replace( # expects string, pattern, replacement
  names(neutral_mhap)# returns the names of the columns
  [seq(2, nc, by = 2)], # creates a sequence of even numbers starting at 2 and ending at nc,
  "\\.\.\.\.[0-9]+$", "") # this cleans up column names. So I don't think this will impact my

# reset the locus names
names(neutral_mhap)[seq(2, nc, by = 2)] <- str_c(loci, "1", sep = ".") #put a .1 at the end of the first
names(neutral_mhap)[seq(3, nc, by = 2)] <- str_c(loci, "2", sep = ".") # put a .2 at the end of the second

## Adaptive
adap_mhap <- read.csv(here("Genomics/Data/Adaptive_genotypes_mhapcode_wide.csv"))

```

Merge the SNP, adaptive, and neutral mhap data frames.

```

merged1 <- merge(adap_mhap,neutral_mhap, by = "indiv.ID")
fullmerge <- merge(snp, merged1, by = "indiv.ID")

```

Save the above data frame for easy access in the repository:

```
write.csv(fullmerge, "Data/merged_ALLgenetics_codewide.csv", row.names = FALSE)
```

Filter out individuals pre-identified in Genomics.PreliminaryAnalysis, the SexID columns. Eventually, we will only want to keep two (ish) of the Omy columns in the adap_mhap data frame.

```
all <- fullmerge %>%
  filter(!indiv.ID %in% c("M145767", "M145542", # matching indivs from scott creek and big cr
                        "M145524", "M145692")) %>%
  dplyr::select(-c(SexID.1, SexID.2))
```

Merge location data from the metadata file with the above data file.

```
populations <- ctmx %>%
  rename(indiv.ID = NMFS_DNA_ID_use) %>%
  filter(!indiv.ID %in% c("M145767", "M145542",
                        "M145524", "M145692")) %>%
  dplyr::select(c(indiv.ID, WATERSHED)) %>%
  left_join(., all, by = "indiv.ID")
```

2. Separate Data Frames for Each Population

```
bigcreek <- populations %>%
  filter(WATERSHED == "Big Creek") %>%
  dplyr::select(-WATERSHED)

scottcreek <- populations %>%
  filter(WATERSHED == "Scott Creek") %>%
  dplyr::select(-WATERSHED)

carmelriver <- populations %>%
  filter(WATERSHED == "Carmel River") %>%
  dplyr::select(-WATERSHED)

napariver <- populations %>%
  filter(WATERSHED == "Napa River") %>%
  dplyr::select(-WATERSHED)
```

```

russianriver <- populations %>%
  filter(WATERSHED == "Russian River")%>%
  dplyr::select(-WATERSHED)

```

3a) Big Sur

```

# make note of the current order of loci in the data set
nc <- ncol(bigcreek)
loci <- str_replace(names(bigcreek)[seq(2, nc, by = 2)], "\\..1$", "")
# then make some long format genotypes
long_genos <- bigcreek %>%
  gather(key = "loc", value = "Allele", -indiv.ID) %>%
  extract( # ChatGPT helped me with this code chunk. had to adapt from CKMR tutorial bc some
    loc,
    into = c("Locus", "gene_copy"),
    regex = "^(.*)\\.\\.([12])$"
  ) %>%
  mutate(Allele = as.character(Allele)) %>%
  mutate(Allele = ifelse(Allele == "0", NA, Allele)) %>%
  rename(Indiv = indiv.ID)

```

Compute allele frequencies.

```

alle_freqs <- long_genos %>%
  count(Locus, Allele) %>%
  group_by(Locus) %>%
  mutate(Freq = n / sum(n),
    Chrom = "Unk",
    Pos = as.integer(factor(Locus, levels = loci))) %>%
  ungroup() %>%
  dplyr::select(Chrom, Pos, Locus, Allele, Freq) %>%
  arrange(Pos, desc(Freq)) %>%
  mutate(AlleIdx = NA,
    LocIdx = NA) %>%
  filter(!is.na(Allele))

# see what that looks like:
alle_freqs

```

```
# A tibble: 665 x 7
  Chrom   Pos Locus      Allele  Freq AlleIdx LocIdx
  <chr> <int> <chr>      <chr>  <dbl> <lg1>  <lg1>
1 Unk     1 Omy_AldA    4      0.673 NA      NA
2 Unk     1 Omy_AldA    3      0.303 NA      NA
3 Unk     2 SH95489_423 3      0.567 NA      NA
4 Unk     2 SH95489_423 1      0.425 NA      NA
5 Unk     3 SH100771_63 4      0.732 NA      NA
6 Unk     3 SH100771_63 1      0.260 NA      NA
7 Unk     4 SH102510_682 2      0.843 NA      NA
8 Unk     4 SH102510_682 1      0.150 NA      NA
9 Unk     5 SH105115_367 2      0.787 NA      NA
10 Unk    5 SH105115_367 3      0.197 NA      NA
# i 655 more rows
```

Reindex the data allele frequency data frame so that ckmr can use it

```
afreqs_ready <- reindex_markers(alle_freqs)
```

Create a ckmr object

```
bigcreek_ckmr <- create_ckmr(
  D = afreqs_ready,
  kappa_matrix = kappas[c("FS", "U"), ],
  ge_mod_assumed = ge_model_TGIE,
  ge_mod_true = ge_model_TGIE,
  ge_mod_assumed_pars_list = list(epsilon = 0.005),
  ge_mod_true_pars_list = list(epsilon = 0.005)
)
bigcreek_ckmr
```

A ckmr object with 254 loci having between 1 and 7 alleles.

Locus names: Unk.Omy_AldA.1, Unk.SH95489_423.2, ... , Unk.tag_id_986_hap1_code.254

Relationships: FS, U

Assumed and true genotyping error models are the same

Simulate genotype pairs and calculate their log probabilities.

```
bigcreek_Qs <- simulate_Qij(bigcreek_ckmr,
  calc_relat = c("FS", "U"),
  sim_relat = c("FS", "U") )
```

Simulating unlinked markers from Y_l_true matrices for relationship: FS

Simulating unlinked markers from Y_l_true matrices for relationship: U

```
bigcreek_Qs
```

```
A Qij object with 10000 reps
simulated with markers unlinked
"sim_relat" relationships: FS, U
"calc_relat" relationships: FS, U
rando_miss_n: 0
```

Compute log-likelihood ratios to view them

```
bigcreek_FS_U_logls <- extract_logls(bigcreek_Qs,
                                     number = c(FS = 1),
                                     denom = c(U = 1))
```

```
bigcreek_FS_U_logls
```

```
# A tibble: 20,000 x 11
```

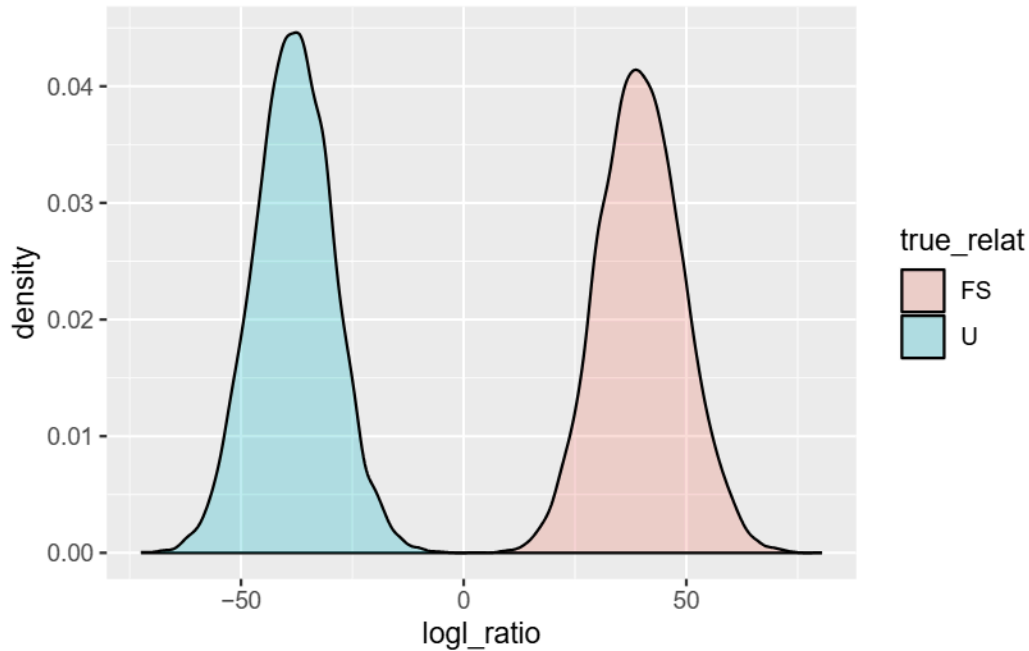
	simtype	P0_sim	rando_miss_n	numer_wts	denom_wts	true_relat	rep	numer_logl
	<chr>	<chr>	<dbl>	<chr>	<chr>	<chr>	<int>	<dbl>
1	unlinked	not_fo~	0	FS=1	U=1	FS	1	-385.
2	unlinked	not_fo~	0	FS=1	U=1	FS	2	-439.
3	unlinked	not_fo~	0	FS=1	U=1	FS	3	-431.
4	unlinked	not_fo~	0	FS=1	U=1	FS	4	-465.
5	unlinked	not_fo~	0	FS=1	U=1	FS	5	-456.
6	unlinked	not_fo~	0	FS=1	U=1	FS	6	-419.
7	unlinked	not_fo~	0	FS=1	U=1	FS	7	-469.
8	unlinked	not_fo~	0	FS=1	U=1	FS	8	-438.
9	unlinked	not_fo~	0	FS=1	U=1	FS	9	-444.
10	unlinked	not_fo~	0	FS=1	U=1	FS	10	-433.

```
# i 19,990 more rows
```

```
# i 3 more variables: denom_logl <dbl>, logl_ratio <dbl>, true_log_prob <dbl>
```

Visualize

```
ggplot(bigcreek_FS_U_logls,
       aes(x = logl_ratio, fill = true_rel)) +
  geom_density(alpha = 0.25)
```



Estimating FPR

```
bigcreek_FS_is <- mc_sample_simple(bigcreek_Qs,
                                   nu = "FS",
                                   de = "U")
```

```
bigcreek_FS_is
```

```
# A tibble: 5 x 10
```

	FNR	FPR	se	num_nonzero_wts	Lambda_star	pstar	mc_method	numerator
	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<chr>	<chr>	<chr>
1	0.01	3.22e-11	3.47e-12	9900	18.5	FS	IS	FS
2	0.05	2.66e-13	1.51e-14	9500	24.7	FS	IS	FS
3	0.1	1.63e-14	6.86e-16	9000	28.0	FS	IS	FS
4	0.2	5.39e-16	2.03e-17	8000	31.7	FS	IS	FS
5	0.3	3.30e-17	1.11e-18	7000	34.7	FS	IS	FS

```
# i 2 more variables: denominator <chr>, true_rel <chr>
```

```
FPR_results <- mc_sample_simple(
  bigcreek_Qs,
  nu = "FS",
  de = "U",
  lambda_stars = seq(5, 20, by = 0.1)
)
```

How many comparisons? Setting acceptable FPR

127 individuals

```
0.1*((127*126)/2)^-1
```

```
[1] 1.249844e-05
```

Setting logLR

Logl ratio that gives largest FPR smaller than 1.249 e-5:

```
bigcreek_FS_is %>%
  filter(FPR > 1.249844e-05) %>%
  arrange(FPR) %>%
  slice(1)
```

```
# A tibble: 0 x 10
# i 10 variables: FNR <dbl>, FPR <dbl>, se <dbl>, num_nonzero_wts <int>,
#   Lambda_star <dbl>, pstar <chr>, mc_method <chr>, numerator <chr>,
#   denominator <chr>, true_relat <chr>
```

Seems like we can set logLR to 18, but this is unclear.

Check for matchers.

```
matchers <- find_close_matching_genotypes(LG = long_genos,
                                          CK = bigcreek_ckmr,
                                          max_mismatch = 6)

matchers
```



```
[1] indiv_1      indiv_2      ind1          ind2          num_mismatch
[6] num_loc
<0 rows> (or 0-length row.names)
```

None!

Compute LogLR for all pairwise comparisons.

```
# Full sib pairs
bigcreek_fs_pairwise_logls <- pairwise_kin_logl_ratios(D1 = long_genos,
                                                    D2 = long_genos,
                                                    CK = bigcreek_ckmr,
                                                    numer = "FS",
                                                    denom = "U",
                                                    num_cores = 1)
```

D1 and D2 are identical: dropping self comparisons and keeping only first instance of each p

```
# note, num_cores is set to 1 because more than that breaks
# it on CRAN checks. But, leave it blank (i.e., don't use the
# num_core option) to, by default, use all
# your cores in parallel on a non-windows machine.
```

```
bigcreek_fs_pairwise_logls %>%
  filter(logl_ratio > 18) %>%
  arrange(desc(logl_ratio))
```

```
# A tibble: 243 x 4
  D2_indiv D1_indiv logl_ratio num_loc
  <chr>    <chr>      <dbl>   <int>
1 M145871 M145872    111.     253
2 M145518 M145533     89.0     253
3 M145532 M145564     82.1     247
4 M145503 M145563     81.8     249
5 M145495 M145509     79.1     253
6 M145534 M145591     76.9     250
7 M145497 M145500     73.2     252
8 M145569 M145867     70.2     253
9 M145530 M145569     69.1     252
10 M145550 M145590     68.9     253
# i 233 more rows
```

```
bigcreek_FS_is %>%
  arrange(Lambda_star) %>%
  dplyr::select(Lambda_star, FPR)
```

```
# A tibble: 5 x 2
  Lambda_star      FPR
      <dbl>    <dbl>
1      18.5 3.22e-11
2      24.7 2.66e-13
3      28.0 1.63e-14
4      31.7 5.39e-16
5      34.7 3.30e-17
```

3b) Scott Creek

```
nc <- ncol(scottcreek)
loci <- str_replace(names(scottcreek)[seq(2, nc, by = 2)], "\\..1$", "")
# then make some long format genotypes
long_genos <- scottcreek %>%
  gather(key = "loc", value = "Allele", -indiv.ID) %>%
  extract( # ChatGPT helped me with this code chunk. had to adapt from CKMR tutorial bc some
    loc,
    into = c("Locus", "gene_copy"),
    regex = "^(.*)\\.\\.([12])$"
  ) %>%
  mutate(Allele = as.character(Allele)) %>%
  mutate(Allele = ifelse(Allele == "0", NA, Allele)) %>%
  rename(Indiv = indiv.ID)
```

Compute allele frequencies.

```
alle_freqs <- long_genos %>%
  count(Locus, Allele) %>%
  group_by(Locus) %>%
  mutate(Freq = n / sum(n),
         Chrom = "Unk",
         Pos = as.integer(factor(Locus, levels = loci))) %>%
  ungroup() %>%
  dplyr::select(Chrom, Pos, Locus, Allele, Freq) %>%
```

```

arrange(Pos, desc(Freq)) %>%
mutate(AlleleIdx = NA,
       LocIdx = NA) %>%
filter(!is.na(Allele))

# see what that looks like:
alle_freqs

```

```

# A tibble: 615 x 7
  Chrom   Pos Locus      Allele  Freq AlleleIdx LocIdx
  <chr> <int> <chr>      <chr>  <dbl> <lg1>    <lg1>
1 Unk     1 Omy_AldA    4      0.891 NA        NA
2 Unk     1 Omy_AldA    3      0.109 NA        NA
3 Unk     2 SH95489_423 3      0.672 NA        NA
4 Unk     2 SH95489_423 1      0.286 NA        NA
5 Unk     3 SH100771_63 4      0.731 NA        NA
6 Unk     3 SH100771_63 1      0.261 NA        NA
7 Unk     4 SH102510_682 2      0.794 NA        NA
8 Unk     4 SH102510_682 1      0.197 NA        NA
9 Unk     5 SH105115_367 2      0.731 NA        NA
10 Unk    5 SH105115_367 3      0.244 NA        NA
# i 605 more rows

```

Reindex the data allele frequency data frame so that ckmr can use it

```
afreqs_ready <- reindex_markers(alle_freqs)
```

Create a ckmr object

```

scottcreek_ckmr <- create_ckmr(
  D = afreqs_ready,
  kappa_matrix = kappas[c("FS", "U"), ],
  ge_mod_assumed = ge_model_TGIE,
  ge_mod_true = ge_model_TGIE,
  ge_mod_assumed_pars_list = list(epsilon = 0.005),
  ge_mod_true_pars_list = list(epsilon = 0.005)
)
scottcreek_ckmr

```

A ckmr object with 254 loci having between 1 and 6 alleles.

Locus names: Unk.Omy_AldA.1, Unk.SH95489_423.2, ... , Unk.tag_id_986_hap1_code.254
 Relationships: FS, U
 Assumed and true genotyping error models are the same

Simulate genotype pairs and calculate their log probabilities.

```
scottcreek_Qs <- simulate_Qij(scottcreek_ckmr,
                             calc_rels = c("FS", "U"),
                             sim_rels = c("FS", "U") )
```

Simulating unlinked markers from Y_l_true matrices for relationship: FS

Simulating unlinked markers from Y_l_true matrices for relationship: U

```
scottcreek_Qs
```

```
A Qij object with 10000 reps
simulated with markers unlinked
"sim_rels" relationships: FS, U
"calc_rels" relationships: FS, U
rando_miss_n: 0
```

Compute log-likelihood ratios to view them

```
scottcreek_FS_U_logls <- extract_logls(scottcreek_Qs,
                                       number = c(FS = 1),
                                       denom = c(U = 1))
```

```
scottcreek_FS_U_logls
```

```
# A tibble: 20,000 x 11
```

	simtype	PO_sim	rando_miss_n	numer_wts	denom_wts	true_relat	rep	numer_logl
	<chr>	<chr>	<dbl>	<chr>	<chr>	<chr>	<int>	<dbl>
1	unlinked	not_fo~	0	FS=1	U=1	FS	1	-412.
2	unlinked	not_fo~	0	FS=1	U=1	FS	2	-439.
3	unlinked	not_fo~	0	FS=1	U=1	FS	3	-411.
4	unlinked	not_fo~	0	FS=1	U=1	FS	4	-409.
5	unlinked	not_fo~	0	FS=1	U=1	FS	5	-430.
6	unlinked	not_fo~	0	FS=1	U=1	FS	6	-403.
7	unlinked	not_fo~	0	FS=1	U=1	FS	7	-406.

```

8  unlinked not_fo~      0 FS=1      U=1      FS      8      -408.
9  unlinked not_fo~      0 FS=1      U=1      FS      9      -391.
10 unlinked not_fo~      0 FS=1      U=1      FS     10      -412.
# i 19,990 more rows
# i 3 more variables: denom_logl <dbl>, logl_ratio <dbl>, true_log_prob <dbl>

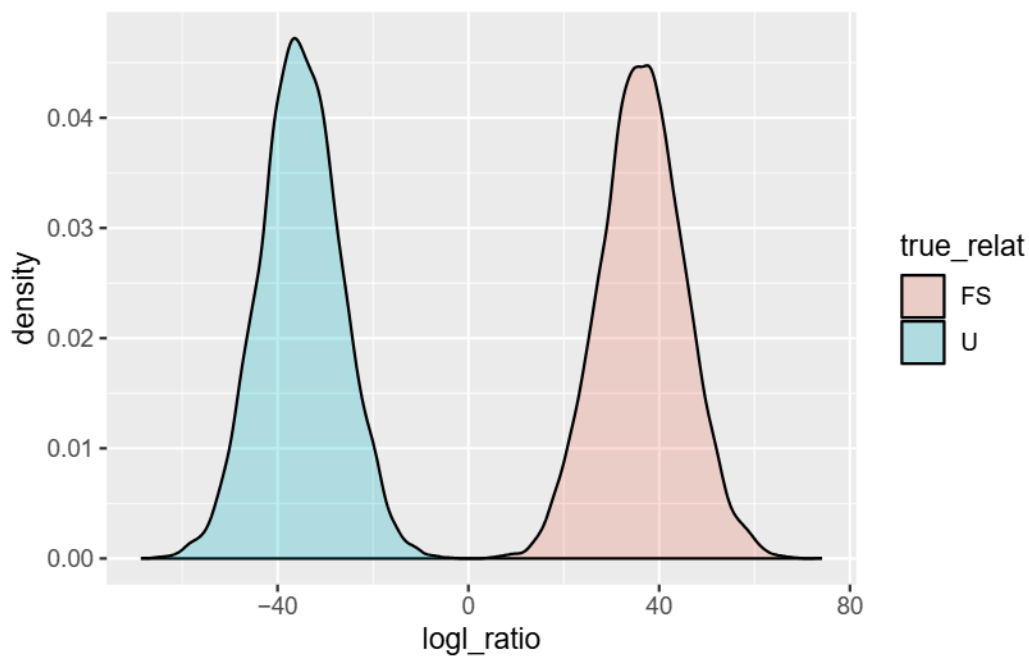
```

Visualize

```

ggplot(scottcreek_FS_U_logls,
       aes(x = logl_ratio, fill = true_rel)) +
  geom_density(alpha = 0.25)

```



3c) Carmel River

```

# make note of the current order of loci in the data set
nc <- ncol(carmelriver)
loci <- str_replace(names(carmelriver)[seq(2, nc, by = 2)], "\\..1$", "")
# then make some long format genotypes
long_genos <- carmelriver %>%
  gather(key = "loc", value = "Allele", -indiv.ID) %>%

```

```

extract( # ChatGPT helped me with this code chunk. had to adapt from CKMR tutorial bc some
  loc,
  into = c("Locus", "gene_copy"),
  regex = "^(.*)\\.[12])$"
) %>%
mutate(Allele = as.character(Allele)) %>%
mutate(Allele = ifelse(Allele == "0", NA, Allele)) %>%
rename(Indiv = indiv.ID)

```

Compute allele frequencies.

```

alle_freqs <- long_genos %>%
  count(Locus, Allele) %>%
  group_by(Locus) %>%
  mutate(Freq = n / sum(n),
         Chrom = "Unk",
         Pos = as.integer(factor(Locus, levels = loci))) %>%
  ungroup() %>%
  dplyr::select(Chrom, Pos, Locus, Allele, Freq) %>%
  arrange(Pos, desc(Freq)) %>%
  mutate(AlleIdx = NA,
         LocIdx = NA) %>%
  filter(!is.na(Allele))

# see what that looks like:
alle_freqs

```

```

# A tibble: 650 x 7
  Chrom   Pos Locus      Allele  Freq AlleIdx LocIdx
  <chr> <int> <chr>      <chr>  <dbl> <lgl>   <lgl>
1 Unk     1 Omy_AldA    4      0.759 NA      NA
2 Unk     1 Omy_AldA    3      0.224 NA      NA
3 Unk     2 SH95489_423 3      0.504 NA      NA
4 Unk     2 SH95489_423 1      0.453 NA      NA
5 Unk     3 SH100771_63 4      0.694 NA      NA
6 Unk     3 SH100771_63 1      0.263 NA      NA
7 Unk     4 SH102510_682 2      0.823 NA      NA
8 Unk     4 SH102510_682 1      0.168 NA      NA
9 Unk     5 SH105115_367 2      0.871 NA      NA
10 Unk    5 SH105115_367 3      0.121 NA      NA
# i 640 more rows

```

Reindex the data allele frequency data frame so that ckmr can use it

```
afreqs_ready <- reindex_markers(alle_freqs)
```

Create a ckmr object

```
carmelriver_ckmr <- create_ckmr(  
  D = afreqs_ready,  
  kappa_matrix = kappas[c("FS", "U"), ],  
  ge_mod_assumed = ge_model_TGIE,  
  ge_mod_true = ge_model_TGIE,  
  ge_mod_assumed_pars_list = list(epsilon = 0.005),  
  ge_mod_true_pars_list = list(epsilon = 0.005)  
)  
carmelriver_ckmr
```

A ckmr object with 254 loci having between 1 and 6 alleles.

Locus names: Unk.0my_AldA.1, Unk.SH95489_423.2, ... , Unk.tag_id_986_hap1_code.254

Relationships: FS, U

Assumed and true genotyping error models are the same

Simulate genotype pairs and calculate their log probabilities.

```
carmelriver_Qs <- simulate_Qij(carmelriver_ckmr,  
  calc_relat = c("FS", "U"),  
  sim_relat = c("FS", "U") )
```

Simulating unlinked markers from Y_l_true matrices for relationship: FS

Simulating unlinked markers from Y_l_true matrices for relationship: U

```
carmelriver_Qs
```

A Qij object with 10000 reps

simulated with markers unlinked

"sim_relat" relationships: FS, U

"calc_relat" relationships: FS, U

rando_miss_n: 0

Compute log-likelihood ratios to view them

```
carmelriver_FS_U_logls <- extract_logls(carmelriver_Qs,
  numer = c(FS = 1),
  denom = c(U = 1))

carmelriver_FS_U_logls
```

```
# A tibble: 20,000 x 11
```

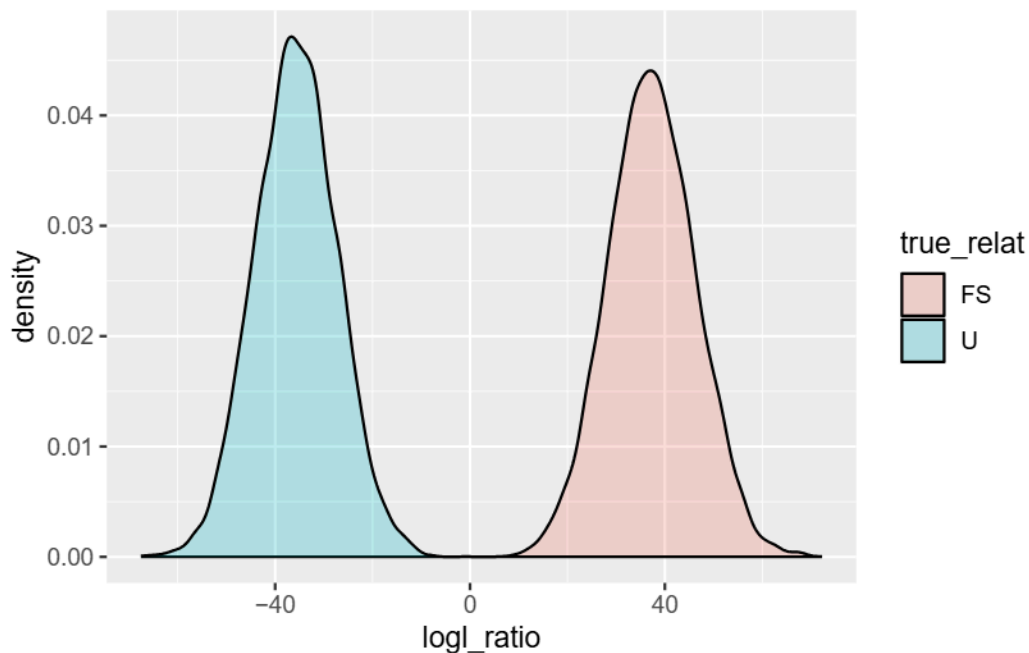
	simtype	PO_sim	rando_miss_n	numer_wts	denom_wts	true_relat	rep	numer_logl
	<chr>	<chr>	<dbl>	<chr>	<chr>	<chr>	<int>	<dbl>
1	unlinked	not_fo~	0	FS=1	U=1	FS	1	-420.
2	unlinked	not_fo~	0	FS=1	U=1	FS	2	-388.
3	unlinked	not_fo~	0	FS=1	U=1	FS	3	-403.
4	unlinked	not_fo~	0	FS=1	U=1	FS	4	-435.
5	unlinked	not_fo~	0	FS=1	U=1	FS	5	-413.
6	unlinked	not_fo~	0	FS=1	U=1	FS	6	-377.
7	unlinked	not_fo~	0	FS=1	U=1	FS	7	-422.
8	unlinked	not_fo~	0	FS=1	U=1	FS	8	-415.
9	unlinked	not_fo~	0	FS=1	U=1	FS	9	-394.
10	unlinked	not_fo~	0	FS=1	U=1	FS	10	-433.

```
# i 19,990 more rows
```

```
# i 3 more variables: denom_logl <dbl>, logl_ratio <dbl>, true_log_prob <dbl>
```

Visualize

```
ggplot(carmelriver_FS_U_logls,
  aes(x = logl_ratio, fill = true_relat)) +
  geom_density(alpha = 0.25)
```

3d) Napa River

```
# make note of the current order of loci in the data set
nc <- ncol(napariver)
loci <- str_replace(names(napariver)[seq(2, nc, by = 2)], "\\..1$", "")
# then make some long format genotypes
long_genos <- napariver %>%
  gather(key = "loc", value = "Allele", -indiv.ID) %>%
  extract( # ChatGPT helped me with this code chunk. had to adapt from CKMR tutorial bc some
    loc,
    into = c("Locus", "gene_copy"),
    regex = "^(.*)\\.\\.([12])$"
  ) %>%
  mutate(Allele = as.character(Allele)) %>%
  mutate(Allele = ifelse(Allele == "0", NA, Allele)) %>%
  rename(Indiv = indiv.ID)
```

Compute allele frequencies.

```

alle_freqs <- long_genos %>%
  count(Locus, Allele) %>%
  group_by(Locus) %>%
  mutate(Freq = n / sum(n),
         Chrom = "Unk",
         Pos = as.integer(factor(Locus, levels = loci))) %>%
  ungroup() %>%
  dplyr::select(Chrom, Pos, Locus, Allele, Freq) %>%
  arrange(Pos, desc(Freq)) %>%
  mutate(AlleIdx = NA,
         LocIdx = NA) %>%
  filter(!is.na(Allele))

# see what that looks like:
alle_freqs

```

```

# A tibble: 565 x 7
  Chrom   Pos Locus      Allele  Freq AlleIdx LocIdx
  <chr> <int> <chr>      <chr> <dbl> <lgl>   <lgl>
1 Unk     1 Omy_AldA      4      0.74 NA      NA
2 Unk     1 Omy_AldA      3      0.26 NA      NA
3 Unk     2 SH95489_423  1      0.71 NA      NA
4 Unk     2 SH95489_423  3      0.29 NA      NA
5 Unk     3 SH100771_63  4      0.59 NA      NA
6 Unk     3 SH100771_63  1      0.41 NA      NA
7 Unk     4 SH102510_682  2      0.73 NA      NA
8 Unk     4 SH102510_682  1      0.27 NA      NA
9 Unk     5 SH105115_367  2      0.57 NA      NA
10 Unk    5 SH105115_367  3      0.41 NA      NA
# i 555 more rows

```

Reindex the data allele frequency data frame so that ckmr can use it

```
afreqs_ready <- reindex_markers(alle_freqs)
```

Create a ckmr object

```

napariver_ckmr <- create_ckmr(
  D = afreqs_ready,
  kappa_matrix = kappas[c("FS", "U"), ],
  ge_mod_assumed = ge_model_TGIE,

```

```

    ge_mod_true = ge_model_TGIE,
    ge_mod_assumed_pars_list = list(epsilon = 0.005),
    ge_mod_true_pars_list = list(epsilon = 0.005)
)
napariver_ckmr

```

A ckmr object with 254 loci having between 1 and 5 alleles.
 Locus names: Unk.Omy_AldA.1, Unk.SH95489_423.2, ... , Unk.tag_id_986_hap1_code.254
 Relationships: FS, U
 Assumed and true genotyping error models are the same

Simulate genotype pairs and calculate their log probabilities.

```

napariver_Qs <- simulate_Qij(napariver_ckmr,
                             calc_rels = c("FS", "U"),
                             sim_rels = c("FS", "U") )

```

Simulating unlinked markers from Y_l_true matrices for relationship: FS

Simulating unlinked markers from Y_l_true matrices for relationship: U

```
napariver_Qs
```

A Qij object with 10000 reps
 simulated with markers unlinked
 "sim_rels" relationships: FS, U
 "calc_rels" relationships: FS, U
 rando_miss_n: 0

Compute log-likelihood ratios to view them

```

napariver_FS_U_logls <- extract_logls(napariver_Qs,
                                       numer = c(FS = 1),
                                       denom = c(U = 1))

napariver_FS_U_logls

```

```
# A tibble: 20,000 x 11
```

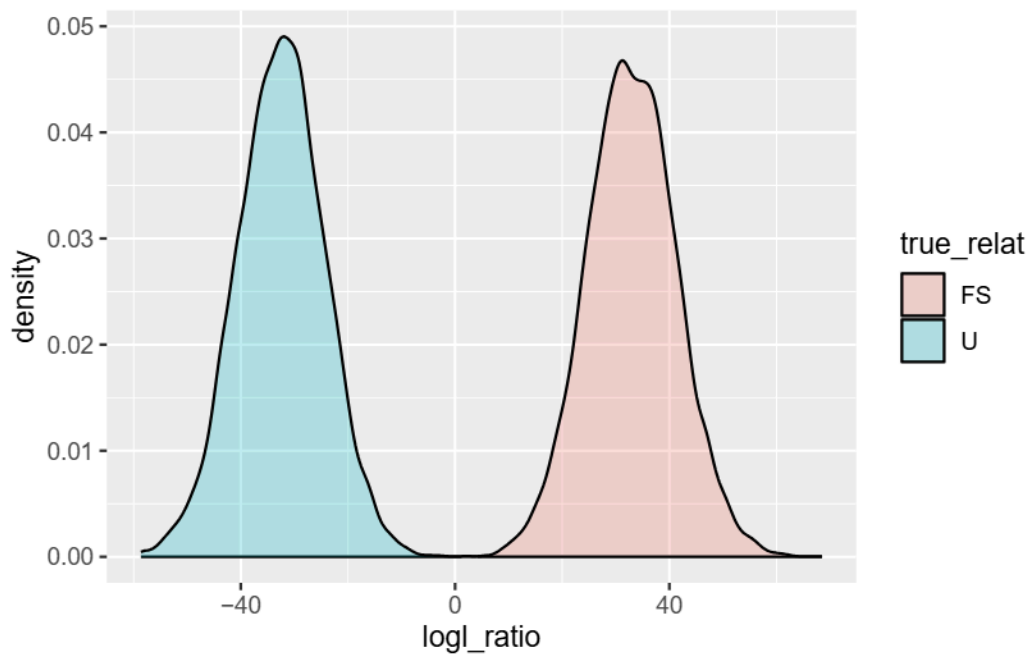
```
  simtype PO_sim  rando_miss_n numer_wts denom_wts true_relat  rep number_logl
  <chr>   <chr>         <dbl> <chr>   <chr>   <chr>         <int>   <dbl>
1 unlinked not_fo~         0 FS=1    U=1    FS             1    -371.
2 unlinked not_fo~         0 FS=1    U=1    FS             2    -400.
3 unlinked not_fo~         0 FS=1    U=1    FS             3    -389.
4 unlinked not_fo~         0 FS=1    U=1    FS             4    -345.
5 unlinked not_fo~         0 FS=1    U=1    FS             5    -356.
6 unlinked not_fo~         0 FS=1    U=1    FS             6    -359.
7 unlinked not_fo~         0 FS=1    U=1    FS             7    -352.
8 unlinked not_fo~         0 FS=1    U=1    FS             8    -347.
9 unlinked not_fo~         0 FS=1    U=1    FS             9    -372.
10 unlinked not_fo~        0 FS=1    U=1    FS            10    -389.
```

```
# i 19,990 more rows
```

```
# i 3 more variables: denom_logl <dbl>, logl_ratio <dbl>, true_log_prob <dbl>
```

Visualize

```
ggplot(napariver_FS_U_logls,
       aes(x = logl_ratio, fill = true_relat)) +
  geom_density(alpha = 0.25)
```



3e) Russian River

```
# make note of the current order of loci in the data set
nc <- ncol(russianriver)
loci <- str_replace(names(russianriver)[seq(2, nc, by = 2)], "\\..1$", "")
# then make some long format genotypes
long_genos <- russianriver %>%
  gather(key = "loc", value = "Allele", -indiv.ID) %>%
  extract( # ChatGPT helped me with this code chunk. had to adapt from CKMR tutorial bc some
    loc,
    into = c("Locus", "gene_copy"),
    regex = "^(..)\\.\\.([12])$"
  ) %>%
  mutate(Allele = as.character(Allele)) %>%
  mutate(Allele = ifelse(Allele == "0", NA, Allele)) %>%
  rename(Indiv = indiv.ID)
```

Compute allele frequencies.

```
alle_freqs <- long_genos %>%
  count(Locus, Allele) %>%
  group_by(Locus) %>%
  mutate(Freq = n / sum(n),
         Chrom = "Unk",
         Pos = as.integer(factor(Locus, levels = loci))) %>%
  ungroup() %>%
  dplyr::select(Chrom, Pos, Locus, Allele, Freq) %>%
  arrange(Pos, desc(Freq)) %>%
  mutate(AlleIdx = NA,
         LocIdx = NA) %>%
  filter(!is.na(Allele))

# see what that looks like:
alle_freqs
```

```
# A tibble: 630 x 7
  Chrom   Pos Locus      Allele   Freq AlleIdx LocIdx
  <chr> <int> <chr>      <chr>   <dbl> <lgl>   <lgl>
1 Unk     1 Omy_AldA      4     0.667  NA      NA
2 Unk     1 Omy_AldA      3     0.319  NA      NA
3 Unk     2 SH95489_423  1     0.5    NA      NA
```

```

4 Unk      2 SH95489_423  3      0.486 NA      NA
5 Unk      3 SH100771_63  1      0.590 NA      NA
6 Unk      3 SH100771_63  4      0.396 NA      NA
7 Unk      4 SH102510_682 2      0.882 NA      NA
8 Unk      4 SH102510_682 1      0.104 NA      NA
9 Unk      5 SH105115_367 2      0.938 NA      NA
10 Unk     5 SH105115_367 3      0.0486 NA      NA
# i 620 more rows

```

Reindex the data allele frequency data frame so that ckmr can use it

```
afreqs_ready <- reindex_markers(alle_freqs)
```

Create a ckmr object

```

russianriver_ckmr <- create_ckmr(
  D = afreqs_ready,
  kappa_matrix = kappas[c("FS", "U"), ],
  ge_mod_assumed = ge_model_TGIE,
  ge_mod_true = ge_model_TGIE,
  ge_mod_assumed_pars_list = list(epsilon = 0.005),
  ge_mod_true_pars_list = list(epsilon = 0.005)
)
russianriver_ckmr

```

A ckmr object with 254 loci having between 1 and 6 alleles.

Locus names: Unk.Omy_AldA.1, Unk.SH95489_423.2, ... , Unk.tag_id_986_hap1_code.254

Relationships: FS, U

Assumed and true genotyping error models are the same

Simulate genotype pairs and calculate their log probabilities.

```

russianriver_Qs <- simulate_Qij(russianriver_ckmr,
  calc_rels = c("FS", "U"),
  sim_rels = c("FS", "U") )

```

Simulating unlinked markers from Y_l_true matrices for relationship: FS

Simulating unlinked markers from Y_l_true matrices for relationship: U

```
russianriver_Qs
```

```
A Qij object with 10000 reps
simulated with markers unlinked
"sim_relat" relationships: FS, U
"calc_relat" relationships: FS, U
rando_miss_n: 0
```

Compute log-likelihood ratios to view them

```
russianriver_FS_U_logls <- extract_logls(russianriver_Qs,
                                         number = c(FS = 1),
                                         denom = c(U = 1))

russianriver_FS_U_logls
```

```
# A tibble: 20,000 x 11
  simtype PO_sim rando_miss_n numer_wts denom_wts true_relat rep number_logl
  <chr>   <chr>         <dbl> <chr>   <chr>   <chr>      <int>    <dbl>
1 unlinked not_fo~         0 FS=1     U=1     FS         1     -415.
2 unlinked not_fo~         0 FS=1     U=1     FS         2     -393.
3 unlinked not_fo~         0 FS=1     U=1     FS         3     -398.
4 unlinked not_fo~         0 FS=1     U=1     FS         4     -389.
5 unlinked not_fo~         0 FS=1     U=1     FS         5     -412.
6 unlinked not_fo~         0 FS=1     U=1     FS         6     -406.
7 unlinked not_fo~         0 FS=1     U=1     FS         7     -422.
8 unlinked not_fo~         0 FS=1     U=1     FS         8     -415.
9 unlinked not_fo~         0 FS=1     U=1     FS         9     -416.
10 unlinked not_fo~         0 FS=1     U=1     FS        10     -415.
# i 19,990 more rows
# i 3 more variables: denom_logl <dbl>, logl_ratio <dbl>, true_log_prob <dbl>
```

Visualize

```
ggplot(russianriver_FS_U_logls,
       aes(x = logl_ratio, fill = true_relat)) +
  geom_density(alpha = 0.25)
```

